MoA Prediction — Fold and drug diagnostics

Companion of `moa-training.ipynb`. It runs in a few minutes, **without GPU and without retraining**, on the files that the training notebook already saved in its output folder. It answers four questions about a drug-aware CV run:

1. **Why is one fold so much worse than the others?** Fold sizes, losses per fold, and how much of the difference comes from very frequent drugs.
2. **Which drugs are the largest, in which fold do they fall, and which targets do they carry?**
3. **Which targets are dominated by a single drug,** and how badly are they predicted when that drug is held out?
4. **Which `HYBRID_MIN_ROWS` threshold gives balanced folds?** Simulated on the real labels in seconds, before spending GPU time on a new run.
5. **How does the gap between CV protocols decompose by drug frequency?** (section 7) Loss on the rows of frequent drugs and on all the other rows, for the row-wise, hybrid and grouped runs, with a bootstrap over drugs.

Inputs

The output folder of a run of `moa-training.ipynb` (attach it as a Dataset, or point `RUN_DIR` to it). Files used: `oof_*.npy`, `oof_targets.npy`, `cv_fold_of_row.npy`, `train_drug_ids.npy`, `metadata.json` (all written by the training notebook, Steps 14.9 and 15.3). Section 7 additionally needs the `oof_*.npy` and `oof_targets.npy` of the row-wise run (`ROW_RUN_DIR`), and optionally of other runs (`EXTRA_RUN_DIRS`). If more than one attached Dataset contains `cv_fold_of_row.npy`, set `RUN_DIR` explicitly: the automatic search stops when it finds several. The notebook only reads these files; the only dependency besides `numpy` and `pandas` is `iterative-stratification` (needed by section 6).

How to read the results

- Everything here is descriptive. A frequent drug being in the worst fold is a *candidate explanation*, not a proof: with 5 folds and a handful of very large drugs, fold composition and fold difficulty are confounded.
- Under `"grouped"` every drug is in a single fold. Under `"hybrid"` the rows of frequent drugs are spread over several folds; the "fold" of such a drug is then the most common one.

In [1]:
```
# Step 1.1 - install the multi-label stratification library (needed only by section 6; internet must be on)
!pip install -q iterative-stratification
```

In [2]:
```python
# Step 1.2 - imports and configuration
import os
import glob
import json
import numpy as np
import pandas as pd
from iterstrat.ml_stratifiers import MultilabelStratifiedKFold

# Folder with the outputs of the training run. None = look for it under /kaggle/input by file name.
RUN_DIR = "/kaggle/input/datasets/marcomerlo37/moa-diagnostics-oof/hybrid"
FREQUENT_MIN_ROWS = 18        # a drug is "frequent" when it has MORE rows than this (same meaning as HYBRID_MIN_ROWS)
TOP_DRUGS = 12                # drugs listed in section 4
MIN_POS_DOMINANT = 50         # targets with at least this many positives are listed in section 5
THRESHOLDS = [6, 12, 18, 30, 60, 100]   # candidate HYBRID_MIN_ROWS values in section 6
N_FOLDS = 5
SEED = 42
CLIP_EPS = 1e-15


def find_input_directory(file_name):
    """The single folder under /kaggle/input that contains `file_name`."""
    matches = glob.glob(f"/kaggle/input/**/{file_name}", recursive=True)
    directories = sorted({os.path.dirname(path) for path in matches})
    if len(directories) != 1:
        raise FileNotFoundError(
            f"Expected exactly one folder containing '{file_name}' under /kaggle/input, "
            f"found {len(directories)}: {directories}. Attach the training-run output as an input, "
            "or set RUN_DIR explicitly.")
    return directories[0]


if RUN_DIR is None:
    RUN_DIR = find_input_directory("cv_fold_of_row.npy")
print(f"Run folder: {RUN_DIR}")
```

```
Run folder: /kaggle/input/datasets/marcomerlo37/moa-diagnostics-oof/hybrid
```

2. Load the run

Loads the saved OOF predictions, labels, fold of every row and drug of every row, and rebuilds the blend with the weights stored in `metadata.json`. Checks that all arrays are aligned.

In [3]:
```python
# Step 2.1 - load
Y = np.load(f"{RUN_DIR}/oof_targets.npy")
```

```
fold_of_row = np.load(f"{RUN_DIR}/cv_fold_of_row.npy")
drug_ids = np.load(f"{RUN_DIR}/train_drug_ids.npy", allow_pickle=True)
with open(f"{RUN_DIR}/metadata.json") as metadata_file:
    metadata = json.load(metadata_file)
target_cols, weights = metadata["target_cols"], metadata["weights"]
oof = {name: np.load(f"{RUN_DIR}/oof_{name}.npy") for name in weights}

n_rows, n_targets = Y.shape
assert len(fold_of_row) == len(drug_ids) == n_rows, "row-level files are not aligned"
assert len(target_cols) == n_targets and all(P.shape == Y.shape for P in oof.values())
assert set(np.unique(fold_of_row)) == set(range(N_FOLDS)), "unexpected fold labels"
oof["blend"] = sum(weights[name] * oof[name] for name in weights)
print(f"CV scheme of the run: {metadata.get('cv_scheme', 'unknown')} | rows {n_rows} | targets {n_targets} | "
      f"drugs {len(np.unique(drug_ids))}")


# Step 2.2 - losses
def loss_matrix(Y, P, eps=CLIP_EPS):
    """Element-wise binary log loss, shape (rows, targets)."""
    P = np.clip(np.asarray(P, dtype=np.float64), eps, 1 - eps)
    return -(Y * np.log(P) + (1 - Y) * np.log(1 - P))


row_loss = {name: loss_matrix(Y, P).mean(axis=1) for name, P in oof.items()}
print("Pooled OOF log loss (rows-weighted): "
      + " | ".join(f"{name} {row_loss[name].mean():.5f}" for name in row_loss))

rows_per_drug = pd.Series(drug_ids).value_counts()
is_frequent = np.isin(drug_ids, rows_per_drug.index[rows_per_drug > FREQUENT_MIN_ROWS])
print(f"Frequent drugs (> {FREQUENT_MIN_ROWS} rows): {int((rows_per_drug > FREQUENT_MIN_ROWS).sum())} drugs, "
      f"{is_frequent.mean():.1%} of the rows")
```

```
CV scheme of the run: hybrid | rows 21948 | targets 206 | drugs 3288
Pooled OOF log loss (rows-weighted): resnet 0.01735 | ft_transformer 0.01744 | catboost 0.01783 | blend 0.01715
Frequent drugs (> 18 rows): 9 drugs, 9.8% of the rows
```

3. Folds: size, composition and loss

One row per fold. `top5_drug_rows_share` is the share of the fold's rows that belong to its five largest drugs. A fold that is both larger and dominated by a few drugs is a candidate explanation for a loss outlier.

In [4]:

```
# Step 3.1 - per-fold table
rows = []
for fold in range(N_FOLDS):
    in_fold = fold_of_row == fold
```

```
    counts = pd.Series(drug_ids[in_fold]).value_counts()
    rows.append({"fold": fold, "rows": int(in_fold.sum()),
                 "size_vs_mean": in_fold.sum() / (n_rows / N_FOLDS) - 1,
                 "drugs": len(counts), "largest_drug": counts.index[0],
                 "largest_drug_rows": int(counts.iloc[0]),
                 "top5_drug_rows_share": counts.iloc[:5].sum() / in_fold.sum(),
                 **{f"loss_{name}": row_loss[name][in_fold].mean() for name in row_loss}})
per_fold = pd.DataFrame(rows)
print(per_fold.round(5).to_string(index=False))
worst_fold = int(per_fold["loss_blend"].idxmax())
others = per_fold.drop(index=worst_fold)
print(f"\nWorst fold (blend): {worst_fold} with {per_fold.loc[worst_fold, 'loss_blend']:.5f}, "
      f"{per_fold.loc[worst_fold, 'loss_blend'] / others['loss_blend'].mean() - 1:+.1%} vs the mean of the other folds")
```

fold	rows	size_vs_mean	drugs	largest_drug	largest_drug_rows	top5_drug_rows_share	loss_resnet	loss_ft_transformer	loss_catboost	loss_blend
0	4386	-0.00082	655	87d714366	144	0.07342	0.01704	0.01723	0.01747	0.01687
1	4356	-0.00765	665	87d714366	143	0.07231	0.01736	0.01734	0.01770	0.01707
2	4371	-0.00424	671	87d714366	144	0.07138	0.01710	0.01715	0.01776	0.01694
3	4383	-0.00150	663	87d714366	144	0.07278	0.01733	0.01737	0.01783	0.01712
4	4452	0.01422	670	87d714366	143	0.06941	0.01791	0.01811	0.01840	0.01773

```
Worst fold (blend): 4 with 0.01773, +4.3% vs the mean of the other folds
```

In [5]:

```
# Step 3.2 - is the outlier fold explained by the frequent drugs?
rows = []
for fold in range(N_FOLDS):
    in_fold = fold_of_row == fold
    freq, rest = in_fold & is_frequent, in_fold & ~is_frequent
    rows.append({"fold": fold,
                 "rows_frequent": int(freq.sum()), "share_frequent": freq.sum() / in_fold.sum(),
                 "loss_frequent": row_loss["blend"][freq].mean() if freq.any() else np.nan,
                 "rows_other": int(rest.sum()),
                 "loss_other": row_loss["blend"][rest].mean(),
                 "loss_all": row_loss["blend"][in_fold].mean()})
frequent_vs_other = pd.DataFrame(rows)
print(frequent_vs_other.round(5).to_string(index=False))

spread_all = frequent_vs_other["loss_all"].max() - frequent_vs_other["loss_all"].min()
spread_other = frequent_vs_other["loss_other"].max() - frequent_vs_other["loss_other"].min()
print(f"\nSpread of the fold loss (max - min): all rows {spread_all:.5f} | excluding frequent drugs {spread_other:.5f}")
if spread_other < 0.5 * spread_all:
    print("-> the spread shrinks a lot once the frequent drugs are excluded: they explain most of the outlier.")
else:
    print("-> the spread persists without the frequent drugs: the outlier is not (only) a frequent-drug effect.")
```

```
 fold  rows_frequent  share_frequent  loss_frequent  rows_other  loss_other  loss_all
    0            438         0.09986        0.00216        3948     0.01850   0.01687
    1            429         0.09848        0.00188        3927     0.01873   0.01707
    2            422         0.09655        0.00201        3949     0.01853   0.01694
    3            430         0.09811        0.00201        3953     0.01876   0.01712
    4            423         0.09501        0.00186        4029     0.01940   0.01773

Spread of the fold loss (max - min): all rows 0.00086 | excluding frequent drugs 0.00090
-> the spread persists without the frequent drugs: the outlier is not (only) a frequent-drug effect.
```

4. The largest drugs

For each of the largest drugs: rows, fold, share of that fold, loss of its rows, and the three targets with most positives among its rows. A large drug whose positives are concentrated in one or two targets makes those targets nearly unlearnable when the drug is held out.

In [6]:
```python
# Step 4.1 - largest drugs
positives = pd.DataFrame(Y, columns=target_cols)
positives["drug"] = drug_ids
positives_per_drug = positives.groupby("drug").sum()

rows = []
for drug in rows_per_drug.index[:TOP_DRUGS]:
    in_drug = drug_ids == drug
    folds = fold_of_row[in_drug]
    main_fold = int(pd.Series(folds).mode().iloc[0])
    top_targets = positives_per_drug.loc[drug].nlargest(3)
    rows.append({"drug": drug, "rows": int(in_drug.sum()), "folds_spanned": len(np.unique(folds)),
                 "main_fold": main_fold,
                 "share_of_main_fold": (in_drug & (fold_of_row == main_fold)).sum() / (fold_of_row == main_fold).sum(),
                 "loss_blend": row_loss["blend"][in_drug].mean(),
                 "top_targets (positives)": ", ".join(f"{t} ({int(v)})" for t, v in top_targets.items() if v > 0)})
print(pd.DataFrame(rows).round(4).to_string(index=False))
```

drug	rows	folds_spanned	main_fold	share_of_main_fold	loss_blend	top_targets (positives)
87d714366	718	5	0	0.0328	0.0006	nfkb_inhibitor (718), proteasome_inhibitor (718)
9f80f3f77	246	5	0	0.0114	0.0031	hmgcr_inhibitor (246)
8b87a7a83	203	5	1	0.0099	0.0009	cdk_inhibitor (203)
5628cb3ee	202	5	3	0.0096	0.0017	egfr_inhibitor (202)
d08af5d4b	196	5	0	0.0091	0.0016	tubulin_inhibitor (196)
292ab2c28	194	5	0	0.0107	0.0021	
d50f18348	186	5	0	0.0087	0.0059	flt3_inhibitor (186), kit_inhibitor (186), pdgfr_inhibitor (186)
d1b47f29d	178	5	4	0.0083	0.0009	raf_inhibitor (178)
67c879e79	19	5	2	0.0011	0.0292	bacterial_50s_ribosomal_subunit_inhibitor (19)
52d1e6f43	18	1	0	0.0041	0.0124	topoisomerase_inhibitor (18)
d488d031d	18	1	4	0.0040	0.0342	estrogen_receptor_antagonist (18)
83a9ea167	18	1	3	0.0041	0.0553	dna_inhibitor (18), thymidylate_synthase_inhibitor (18)

5. Targets dominated by a single drug

`top_drug_share` is the share of the target's positives that come from its single most frequent drug. `train_pos_when_top_drug_held_out` is how many positives of the target remain in the training part of the fold where that drug is validated. `expected/observed` is the sum of the blend's predicted probabilities divided by the observed positives (1 = calibrated, ≪ 1 = under-predicted).

A high `top_drug_share`, few remaining training positives and `expected/observed` far below 1 together describe a target that the drug-aware CV *cannot* learn in that fold. If the test set contains that drug, the real situation is different.

In [7]:
```
# Step 5.1 - dominated targets
n_pos = Y.sum(axis=0)
rows = []
for t, name in enumerate(target_cols):
    if n_pos[t] < MIN_POS_DOMINANT:
        continue
    per_drug = positives_per_drug[name]
    top_drug = per_drug.idxmax()
    top_fold = int(pd.Series(fold_of_row[drug_ids == top_drug]).mode().iloc[0])
    rows.append({"target": name, "n_pos": int(n_pos[t]), "n_drugs": int((per_drug > 0).sum()),
                 "top_drug": top_drug, "top_drug_share": per_drug.max() / n_pos[t],
                 "train_pos_when_top_drug_held_out": int(n_pos[t] - Y[fold_of_row == top_fold, t].sum()),
                 "expected/observed": oof["blend"][:, t].sum() / n_pos[t]})
dominated = pd.DataFrame(rows).sort_values("top_drug_share", ascending=False)
print(dominated.head(25).round(3).to_string(index=False))

heavy = dominated["top_drug_share"] >= 0.5
print(f"\nTargets with >= {MIN_POS_DOMINANT} positives: {len(dominated)}; "
      f"{int(heavy.sum())} have >= 50% of their positives from one drug")
if heavy.any() and (~heavy).any():
```

```
    print(f"Median expected/observed: dominated {dominated.loc[heavy, 'expected/observed'].median():.2f} | "
          f"others {dominated.loc[~heavy, 'expected/observed'].median():.2f}")
```

target	n_pos	n_drugs	top_drug	top_drug_share	train_pos_when_top_drug_held_out	expected/observed
proteasome_inhibitor	726	4	87d714366	0.989	581	1.031
hmgcr_inhibitor	283	7	9f80f3f77	0.869	227	0.929
nfkb_inhibitor	832	19	87d714366	0.863	665	0.996
raf_inhibitor	223	9	d1b47f29d	0.798	186	0.993
kit_inhibitor	273	17	d50f18348	0.681	228	0.828
flt3_inhibitor	279	17	d50f18348	0.667	223	0.862
pdgfr_inhibitor	297	21	d50f18348	0.626	246	0.814
tubulin_inhibitor	316	22	d08af5d4b	0.620	252	1.047
egfr_inhibitor	336	25	5628cb3ee	0.601	270	0.945
cdk_inhibitor	340	24	8b87a7a83	0.597	286	1.028
insulin_sensitizer	51	8	11f66c124	0.275	31	0.883
bacterial_50s_ribosomal_subunit_inhibitor	80	12	67c879e79	0.238	69	1.013
radiopaque_medium	56	8	88c37c66a	0.232	37	1.178
adenosine_receptor_agonist	54	8	2688e0167	0.222	36	1.018
leukotriene_receptor_antagonist	62	9	d149a15f5	0.210	43	1.031
mek_inhibitor	72	11	1b09b35f0	0.167	53	1.041
hiv_inhibitor	72	11	fe8625cad	0.167	48	1.032
immunosuppressant	73	11	ae0c59f9b	0.164	49	0.976
antioxidant	73	11	95a5b4f7e	0.164	55	1.084
anesthetic_-_local	80	13	10dbb5b2a	0.162	55	1.059
membrane_integrity_inhibitor	74	11	472eaa5fd	0.162	50	1.061
glutamate_receptor_agonist	74	12	de7b27dbc	0.162	50	1.103
opioid_receptor_antagonist	96	14	a7c2673c1	0.146	70	1.006
topoisomerase_inhibitor	127	18	52d1e6f43	0.142	97	1.063
fgfr_inhibitor	50	10	9288cf822	0.140	37	0.803

```
Targets with >= 50 positives: 87; 10 have >= 50% of their positives from one drug
Median expected/observed: dominated 0.97 | others 1.01
```

6. Choosing `HYBRID_MIN_ROWS`

For each candidate threshold the hybrid split is simulated on the real labels (drugs with *more* rows than the threshold are split row by row, the others are kept whole), exactly as `make_split_groups` / `grouped_multilabel_splits` do in the training notebook (the two functions are copied here). The first line, "pure grouped", is the protocol of the run being analysed.

- `max_size_dev` : largest deviation of a fold's size from the mean (the training notebook warns above 15%).
- `val_rows_seen_drug` : average share of validation rows whose drug is also in the training fold. Row-wise CV is ~100%, pure grouped 0%. The lower the threshold, the closer the CV gets to the optimistic row-wise one.
- `targets_without_pos_in_val` : average number of targets with no positive in a validation fold.

Pick the **largest** threshold that brings `max_size_dev` to an acceptable level (a few %): it keeps as much of the drug-aware protocol as possible. This is a trade-off, not an optimum: neither extreme can be shown to be the faithful one.

In [8]:
```
# Step 6.1 - simulate hybrid splits for several thresholds (functions copied from moa-training.ipynb, Step 2.5)
def grouped_multilabel_splits(Y, groups, n_splits, seed):
    unique_groups, group_of_row = np.unique(groups, return_inverse=True)
    group_labels = np.zeros((len(unique_groups), Y.shape[1]), dtype=np.float64)
    np.maximum.at(group_labels, group_of_row, Y)
    splitter = MultilabelStratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)
    splits = []
    for _, val_groups in splitter.split(np.zeros((len(unique_groups), 1)), group_labels):
        is_val = np.isin(group_of_row, val_groups)
        train_idx, val_idx = np.where(~is_val)[0], np.where(is_val)[0]
        assert set(groups[train_idx]).isdisjoint(groups[val_idx]), "a group is split across folds"
        splits.append((train_idx, val_idx))
    return splits


def make_hybrid_groups(drug_ids, min_rows):
    _, drug_of_row, rows_per_drug = np.unique(drug_ids, return_inverse=True, return_counts=True)
    frequent = rows_per_drug[drug_of_row] > min_rows
    row_groups = np.char.add("row_", np.arange(len(drug_ids)).astype(str))
    return np.where(frequent, row_groups, drug_ids), frequent


rows = []
for min_rows in [None, *THRESHOLDS]:
    if min_rows is None:
        groups, frequent = drug_ids, np.zeros(n_rows, dtype=bool)
    else:
        groups, frequent = make_hybrid_groups(drug_ids, min_rows)
    splits = grouped_multilabel_splits(Y, groups, N_FOLDS, SEED)
    sizes = np.array([len(val_idx) for _, val_idx in splits])
    rows.append({
        "scheme": "pure grouped" if min_rows is None else f"hybrid, > {min_rows} rows",
        "rows_split_by_row": frequent.mean(),
        "fold_sizes": sizes.tolist(),
        "max_size_dev": np.abs(sizes / sizes.mean() - 1).max(),
        "val_rows_seen_drug": np.mean([np.isin(drug_ids[va], drug_ids[tr]).mean() for tr, va in splits]),
        "targets_without_pos_in_val": np.mean([(Y[va].sum(axis=0) == 0).sum() for _, va in splits]),
    })
threshold_table = pd.DataFrame(rows)
print(threshold_table.round(3).to_string(index=False))
```

scheme	rows_split_by_row	fold_sizes	max_size_dev	val_rows_seen_drug	targets_without_pos_in_val
pure grouped	0.000	[3987, 4136, 5512, 3933, 4380]	0.256	0.000	35.2
hybrid, > 6 rows	0.220	[4380, 4349, 4539, 4388, 4292]	0.034	0.220	29.8
hybrid, > 12 rows	0.119	[4374, 4487, 4253, 4347, 4487]	0.031	0.119	34.6
hybrid, > 18 rows	0.098	[4386, 4356, 4371, 4383, 4452]	0.014	0.098	35.0
hybrid, > 30 rows	0.097	[4348, 4268, 4438, 4463, 4431]	0.028	0.097	35.0
hybrid, > 60 rows	0.097	[4348, 4268, 4438, 4463, 4431]	0.028	0.097	35.0
hybrid, > 100 rows	0.097	[4348, 4268, 4438, 4463, 4431]	0.028	0.097	35.0

7. How does the gap between protocols decompose by drug frequency?

The three CV protocols differ only in how they treat drugs: row-wise CV lets every drug be seen in training, grouped CV lets none be, hybrid CV lets only the frequent ones (more than `FREQUENT_MIN_ROWS` rows, which must equal the `HYBRID_MIN_ROWS` of the hybrid run). This section splits the loss of each run into the rows of frequent drugs and all the other rows, so that the gap between protocols can be attributed:

- **Frequent rows:** where hybrid and row-wise CV agree with each other and differ from grouped CV.
- **Other rows:** a difference here, row-wise vs hybrid, is the cost of grouping the ordinary drugs. It comes with a bootstrap interval that resamples **drugs**, not rows, and is paired (same rows in both runs).

It also derives which share of the test rows would have to belong to frequent drugs seen in training to match the Kaggle private score, given the losses measured in the hybrid run. That number is a **back-of-the-envelope inference**, not an estimate: it assumes that the test losses on seen and unseen rows equal the CV ones, and the Kaggle score is a single noisy measurement, so the sensitivity to a ±5% error is printed next to it.

In [9]:
```
# Step 7.1 - runs to compare
# Folder with oof_*.npy and oof_targets.npy of the row-wise run (the REFERENCE_OOF_DIR of the training notebook).
ROW_RUN_DIR = "/kaggle/input/datasets/marcomerlo37/moa-diagnostics-oof/row"      # R3
# Other runs to include, label -> folder (for example {"grouped": "/kaggle/input/.../moa-diagnostics-dataset"}).
# The run in RUN_DIR is added automatically, under the CV scheme recorded in its metadata.json.
EXTRA_RUN_DIRS = {"grouped": "/kaggle/input/datasets/marcomerlo37/moa-diagnostics-oof/grouped"}
# Blend weights for folders without metadata.json (documented weights of the row-wise run)
REFERENCE_BLEND_WEIGHTS = {"resnet": 0.4529, "ft_transformer": 0.3746, "catboost": 0.1725}
KAGGLE_PRIVATE_SCORE = 0.01634     # score on all test rows, late submission of the row-wise models
KAGGLE_PUBLIC_SCORE = 0.01865
TEST_TREATED_SHARE = 0.910         # treated share of the public test set; approximate for the private part
N_BOOTSTRAP = 300


def load_run_losses(folder):
    # Row-level loss of each model and of the blend for one run; its labels must match RUN_DIR's
    targets = np.load(f"{folder}/oof_targets.npy")
    assert targets.shape == Y.shape and (targets == Y).all(), \
```

```
        f"{folder}: oof_targets.npy does not match the labels of RUN_DIR (different rows or order)"
    metadata_path = f"{folder}/metadata.json"
    if os.path.exists(metadata_path):
        with open(metadata_path) as metadata_file:
            run_weights = json.load(metadata_file)["weights"]
    else:
        run_weights = REFERENCE_BLEND_WEIGHTS
        print(f"  {folder}: no metadata.json, using REFERENCE_BLEND_WEIGHTS")
    preds = {name: np.load(f"{folder}/oof_{name}.npy") for name in run_weights}
    preds["blend"] = sum(run_weights[name] * preds[name] for name in run_weights)
    return {name: loss_matrix(Y, P).mean(axis=1) for name, P in preds.items()}


run_dirs = {}
if ROW_RUN_DIR is not None:
    run_dirs["row"] = ROW_RUN_DIR
for label, folder in EXTRA_RUN_DIRS.items():
    assert label not in run_dirs, f"duplicate run label: {label}"
    run_dirs[label] = folder
main_label = metadata.get("cv_scheme", "this run")
assert main_label not in run_dirs, f"RUN_DIR has CV scheme '{main_label}', which is already in the list"
run_dirs[main_label] = RUN_DIR

print("Runs:", {label: os.path.basename(folder.rstrip("/")) for label, folder in run_dirs.items()})
run_row_loss = {label: load_run_losses(folder) for label, folder in run_dirs.items()}
if "row" not in run_row_loss:
    print("ROW_RUN_DIR is not set: the comparison with the row-wise run is skipped.")
```

```
Runs: {'row': 'row', 'grouped': 'grouped', 'hybrid': 'hybrid'}
```

In [10]:

```
# Step 7.2 - loss on frequent vs other rows, and paired comparison between runs
share_frequent = float(is_frequent.mean())
print(f"Frequent drugs (> {FREQUENT_MIN_ROWS} rows): {int((rows_per_drug > FREQUENT_MIN_ROWS).sum())} drugs, "
      f"{share_frequent:.1%} of the rows\n")

rows = []
for label, losses in run_row_loss.items():
    for name, row_l in losses.items():
        rows.append({"run": label, "model": name, "loss_all": row_l.mean(),
                     "loss_frequent": row_l[is_frequent].mean(), "loss_other": row_l[~is_frequent].mean()})
by_subset = pd.DataFrame(rows)
print("Blend, by subset of rows:")
print(by_subset[by_subset["model"] == "blend"].drop(columns="model").round(5).to_string(index=False))
print("\nLoss on the OTHER (non-frequent) rows, by model and run:")
print(by_subset.pivot(index="model", columns="run", values="loss_other").round(5).to_string())


def paired_bootstrap_ci(diff_per_row, mask, drug_ids, n_boot=N_BOOTSTRAP, seed=SEED):
```

```
    # 95% interval of the mean per-row difference over `mask`, resampling DRUGS (rows of a drug are correlated)
    unique_drugs, drug_of_row = np.unique(drug_ids, return_inverse=True)
    numerator = np.bincount(drug_of_row, weights=np.where(mask, diff_per_row, 0.0), minlength=len(unique_drugs))
    denominator = np.bincount(drug_of_row, weights=mask.astype(float), minlength=len(unique_drugs))
    rng = np.random.default_rng(seed)
    draws = rng.multinomial(len(unique_drugs), np.full(len(unique_drugs), 1.0 / len(unique_drugs)), size=n_boot)
    estimates = (draws @ numerator) / (draws @ denominator)
    return tuple(np.percentile(estimates, [2.5, 97.5]))


if "row" in run_row_loss:
    print("\nDecomposition of each run's difference from the row-wise run (blend, same rows):")
    for label, losses in run_row_loss.items():
        if label == "row":
            continue
        diff = losses["blend"] - run_row_loss["row"]["blend"]
        d_freq, d_other = diff[is_frequent].mean(), diff[~is_frequent].mean()
        low, high = paired_bootstrap_ci(diff, ~is_frequent, drug_ids)
        print(f"  {label:10s} total {diff.mean():+.5f} = frequent rows {share_frequent * d_freq:+.5f} "
              f"+ other rows {(1 - share_frequent) * d_other:+.5f} | "
              f"other rows: {d_other:+.5f} per row [{low:+.5f}, {high:+.5f}] "
              f"({d_other / run_row_loss['row']['blend'][~is_frequent].mean():+.1%} of the row-wise loss on them)")
    print("  'other rows' is the cost of grouping the ordinary drugs; 'frequent rows' is what hybrid keeps "
          "and pure grouping removes.")

# Which share of the test rows would have to be frequent drugs seen in training to match the Kaggle score?
if "hybrid" in run_row_loss:
    blend_loss = run_row_loss["hybrid"]["blend"]
    seen_loss, unseen_loss = blend_loss[is_frequent].mean(), blend_loss[~is_frequent].mean()
    print(f"\nHybrid run: loss on frequent (seen) rows {seen_loss:.5f} | on other (unseen) rows {unseen_loss:.5f}")
    print("Share p of test rows from seen frequent drugs that reproduces a Kaggle score "
          "[ (unseen - target) / (unseen - seen) ], treated-rows scale:")
    for score_name, score in (("private", KAGGLE_PRIVATE_SCORE), ("public", KAGGLE_PUBLIC_SCORE)):
        target = score / TEST_TREATED_SHARE
        implied = [(unseen_loss - target * (1 + rel)) / (unseen_loss - seen_loss) for rel in (-0.05, 0.0, 0.05)]
        print(f"  {score_name:7s} target {target:.5f}: p = {implied[1]:+.1%}  (score -5% / +5%: "
              f"{implied[0]:+.1%} / {implied[2]:+.1%}); share of such rows in the training set: {share_frequent:.1%}")
    print("p < 0 means the score is above what an all-unseen test would give: frequent drugs cannot explain it. "
          "Inference only: it assumes test losses equal the CV ones and ignores the noise of the Kaggle score.")
```

```
Frequent drugs (> 18 rows): 9 drugs, 9.8% of the rows

Blend, by subset of rows:
    run  loss_all  loss_frequent  loss_other
    row   0.01595        0.00165     0.01750
grouped   0.01921        0.02260     0.01884
 hybrid   0.01715        0.00198     0.01879

Loss on the OTHER (non-frequent) rows, by model and run:
run             grouped   hybrid      row
model
blend           0.01884  0.01879  0.01750
catboost        0.01931  0.01932  0.01841
ft_transformer  0.01913  0.01914  0.01804
resnet          0.01911  0.01904  0.01761

Decomposition of each run's difference from the row-wise run (blend, same rows):
  grouped    total +0.00326 = frequent rows +0.00204 + other rows +0.00121 | other rows: +0.00134 per row [+0.00120, +0.00148] (+7.7% of the row-wise
loss on them)
  hybrid     total +0.00119 = frequent rows +0.00003 + other rows +0.00116 | other rows: +0.00129 per row [+0.00116, +0.00143] (+7.4% of the row-wise
loss on them)
  'other rows' is the cost of grouping the ordinary drugs; 'frequent rows' is what hybrid keeps and pure grouping removes.

Hybrid run: loss on frequent (seen) rows 0.00198 | on other (unseen) rows 0.01879
Share p of test rows from seen frequent drugs that reproduces a Kaggle score [ (unseen - target) / (unseen - seen) ], treated-rows scale:
  private target 0.01796: p = +4.9%  (score -5% / +5%: +10.3% / -0.4%); share of such rows in the training set: 9.8%
  public  target 0.02049: p = -10.2%  (score -5% / +5%: -4.1% / -16.3%); share of such rows in the training set: 9.8%
p < 0 means the score is above what an all-unseen test would give: frequent drugs cannot explain it. Inference only: it assumes test losses equal the
CV ones and ignores the noise of the Kaggle score.
```